How GetEvents Works

A walk-through of the system in plain language — what each part does, and how one piece of information travels from a raw web page to your customer's API response.

THE ONE-SENTENCE VERSION

> We run something like a **global newsroom for events**. Hundreds of correspondents around the world (data sources) file raw, messy reports. Before a single one of those reports reaches a reader — your API customers — it passes through a copy desk that checks it, translates it into a common format, catches duplicate reports of the same story, confirms the facts, adds a map location, files it under the right topic, estimates how big a deal it is, and only then sends it out on the wire.

WHY NOT JUST A WEBSITE?

A ticket-selling website shows events to people. That is not what this is. This is a **data supplier** — other companies' software (a travel app, a hotel pricing engine, a staffing planner) will connect to our API and pull events automatically, the same way a weather app pulls a forecast. Our customer is a computer program, not a person browsing listings — which is exactly why the quality of the cleanup work in the middle matters so much more than how anything looks.

THE ROUTE A PIECE OF INFORMATION TAKES

Every event, from every source, walks the same 11 stops before it's allowed into the system customers actually see. No exceptions — raw information is never handed straight to a customer.

01 **Correspondent**
THE CONNECTOR

02 **The inbox**
RAW EVENT STORAGE

03 **The translator**
NORMALIZER

04 **The duplicate-catcher**
DEDUPLICATOR

05 **The fact-checker**
ENTITY RESOLUTION

06 **The map desk**
GEOCODER

07 **The filing clerk**
CATEGORY CLASSIFIER

08 **The crowd forecaster**
ATTENDANCE ESTIMATOR

09 **The ratings desk**
IMPACT SCORING

10 **The master file**
CANONICAL EVENT DATABASE

11 **The wire out**
API

EACH STOP, EXPLAINED

01 Correspondent

`also called: the Connector`

A small, dedicated piece of code that knows how to talk to exactly **one** outside source — Ticketmaster, a city's tourism board, a university calendar, a weather agency. Its only job is to go get that source's events and hand them over in a standard shape. We can have hundreds of these running side by side, and adding a new one for a new source is a small, contained job, not a rebuild of anything else.

> **Example:** the Ticketmaster correspondent checks Ticketmaster for new concerts; the NOAA correspondent checks for severe weather alerts. Neither knows the other exists.

02 The inbox

`also called: Raw Event Storage`

Everything that comes in gets dropped here **exactly as received** — untouched, unedited — before anyone does anything with it. Think of it as a mailroom that photographs every letter before it's opened. This matters because if something goes wrong three steps later, we can always go back to what actually arrived and see the original.

> **Rule:** nothing from this inbox is ever shown to a customer directly. It only ever moves forward, one desk at a time.

03 The translator

`also called: the Normalizer`

Every source describes an event differently — different date formats, different field names, different units. This desk rewrites every incoming report into one common, consistent shape, so everything downstream can be built once instead of once per source.

> **Example:** "7/8/26," "08-Jul-2026," and "2026-07-08T00:00" all become the same one format.

04 The duplicate-catcher

`also called: the Deduplicator`

The same concert is often reported by three or four different sources at once. This desk notices that and merges them into a single story instead of showing the same event three times.

> **Example:** Ticketmaster, Songkick, and the venue's own website all list "Coldplay, Wembley, Aug 12" — this desk recognizes it's one event, not three.

05 The fact-checker

`also called: Entity Resolution`

Confirms that the *things* mentioned in an event — the venue, the artist, the organizer — are recognized as the same real-world thing every time, no matter how they were spelled or abbreviated in the original report.

> **Example:** "MSG," "Madison Square Garden," and "The Garden" are all confirmed as the one venue.

06 The map desk

`also called: the Geocoder`

Attaches an exact map location, timezone, and administrative area (city, region, country) to the event, even if the original report only gave a venue name or a rough address.

> **Example:** "The O2, London" becomes a precise map pin, "Europe/London" timezone, and "Greenwich, London, UK."

07 The filing clerk

`also called: the Category Classifier`

Reads the event and files it under the right topic — concert, sporting event, conference, festival, public holiday, severe weather, disaster, health warning, and so on — so customers can ask for exactly the kind of event they care about.

> **Example:** a wildfire alert gets filed under "disasters," not "community events."

08 The crowd forecaster

`also called: the Attendance Estimator`

Estimates how many people are likely to show up, using venue size, past turnout for similar events, and how popular the artist, team, or subject is — even when the original source didn't say.

> **Example:** a stadium concert by a top-selling act gets estimated near full capacity; a local farmers' market gets a modest estimate.

09 The ratings desk

`also called: Impact Scoring`

Turns everything gathered so far into one "how much does this matter" number — factoring in expected crowd size, category, and what else is happening nearby at the same time. This is the number a hotel-pricing or staffing customer actually wants.

> **Example:** a 60,000-seat stadium concert on a Friday scores far higher impact than a 200-person community talk the same day.

10 The master file

`also called: the Canonical Event Database`

The single, clean, agreed-upon record for that event — the one version of the truth, after every desk above has done its job. If the event changes later (rescheduled, cancelled), this is the record that gets updated, not a new one created.

> **Rule:** this is the only place a customer-facing answer is allowed to come from.

11 The wire out

`also called: the API`

The delivery pipe. Customers' software connects here and asks questions — "what's happening in Austin next weekend," "any disasters near this warehouse this week" — and gets back clean, scored, categorized answers pulled straight from the master file.

FOLLOW ONE CONCERT THROUGH THE WHOLE SYSTEM

A concrete walk-through, start to finish, for a single real-world listing.

SOURCE	Ticketmaster's page lists **"Coldplay — Wembley Stadium — 8/12/26."**
01 · CORRESPONDENT	The Ticketmaster connector fetches this listing on its usual schedule.
02 · INBOX	The exact raw listing is saved untouched.
03 · TRANSLATOR	The date becomes **2026-08-12**; fields are relabeled into our standard event format.
04 · DUPLICATE-CATCHER	Songkick and the Wembley website both reported the same show — all three are merged into one event.
05 · FACT-CHECKER	"Wembley Stadium" is confirmed as our one master record for that venue; "Coldplay" is confirmed as our one master record for that artist.

06 · MAP DESK	Location is set to an exact map pin in Wembley, London, timezone **Europe/London**.
07 · FILING CLERK	Filed under **Concerts**.
08 · CROWD FORECASTER	Estimated at roughly **90,000** attendees, near the stadium's full capacity.
09 · RATINGS DESK	Given a **high impact score** — large crowd, major road closures and hotel demand expected nearby.
10 · MASTER FILE	Saved as the one canonical record for this concert.
11 · WIRE OUT	A hotel-pricing customer's software asks the API "what's happening near Wembley in August" and gets this event back, fully formed.

HOW THE DESKS TALK TO EACH OTHER

Each desk doesn't wait around asking the next one "are you ready yet?" Instead, the moment a desk finishes its job, it drops a note on a shared line saying "I'm done, here's the result" — and whichever desk is next picks it up automatically, whenever it's ready. Nobody blocks anybody else.

- → This means every desk can work at its own pace, and if the map desk gets busy, it doesn't slow down the filing clerk or the duplicate-catcher.
- → If a note gets dropped and nobody picks it up (say, an outside mapping service is briefly down), it's held safely and retried automatically — nothing quietly disappears.
- → A separate scheduler handles a different concern entirely: deciding *when* each correspondent should go check its source again — every 15 minutes, once a day, whatever fits that source.

WHY IT'S BUILT THIS WAY

The whole point of separating these into distinct desks, each doing exactly one job, is what makes scaling to hundreds or thousands of sources realistic instead of a rebuild every time:

- → Adding source number 501 means writing one small correspondent — nothing else in the system changes.
- → If the duplicate-catcher needs to get smarter, that's the only desk that has to change.

→ If one correspondent breaks because a website redesigned its page, only that one source is affected — everything else keeps running.

→ Raw information is never shown to a customer directly, so a messy or wrong source can never leak straight through to someone paying for clean data.

QUICK-REFERENCE GLOSSARY

PLAIN-ENGLISH NAME	TECHNICAL NAME	IN ONE LINE
Correspondent	`Connector`	Code that fetches events from one specific outside source.
Inbox	`Raw Event Storage`	Untouched copy of everything received, kept before any cleanup.
Translator	`Normalizer`	Rewrites every source's format into one common shape.
Duplicate-catcher	`Deduplicator`	Merges multiple reports of the same real event into one.
Fact-checker	`Entity Resolution`	Confirms venues, artists, and organizers are the same known entity.
Map desk	`Geocoder`	Adds exact location, timezone, and administrative area.
Filing clerk	`Category Classifier`	Sorts the event into the right topic/category.
Crowd forecaster	`Attendance Estimator`	Estimates expected turnout.
Ratings desk	`Impact Scoring`	Scores how much the event will affect its surroundings.
Master file	`Canonical Event Database`	The single clean, authoritative record for the event.
Wire out	`API`	How customers' software asks for and receives event data.

PLAIN-ENGLISH NAME	TECHNICAL NAME	IN ONE LINE
Plugin	`Connector plugin`	Why a connector is built so it can be added without touching the rest of the system.